CodeArts Best Practices

Speaker Script — 23 Slides

A walkthrough for presenting the Phoenix Mall reference project

Runtime: 25–30 minutes · Leaves room for Q&A

Prepared by Reshoan · CodeArts Implementation

Slide 1 — Title: CodeArts Best Practices

> "Thanks everyone for making the time. Today I want to walk us through what implementing CodeArts looks like in practice — not as a marketing tour of features, but as a concrete, end-to-end example of how a real team would use it.

> What you'll see is based on the official Huawei CodeArts best-practices guide, walked through using a sample project called Phoenix Mall. By the end you should have a clear mental model of how we plan work in CodeArts, how code moves from a developer's laptop to a running cluster, how we keep it secure, and how nine different teams collaborate inside it.

> But before any of that — let me spend a couple of minutes on **what CodeArts actually is**, because I know not everyone here has worked with it directly. Let's go to the next slide."

▸ ADVANCE

Slide 2 — What is CodeArts?

"**CodeArts is Huawei Cloud's integrated DevOps platform.** Think of it as a single SaaS suite that bundles every tool a software team needs to take an idea from a backlog item to a running production deployment.

The pitch is straightforward: instead of stitching together five or six different vendors with integration plugins and brittle glue, you get one platform with native traceability from work item, to commit, to scan, to build, to deploy, to test case — all under one identity and one permission model.

What's on screen is the ten services that make up the suite. Reading across:

Req is requirements and agile project management — roughly a Jira equivalent.

Repo is Git-based source code hosting — think GitHub or GitLab.

Check does static code analysis with quality gates — think SonarQube.

Build is CI build automation — think Jenkins or GitHub Actions.

Artifact is the package and binary repository — think Artifactory or Nexus.

Deploy handles deployment to both VMs and Kubernetes — think AWS CodeDeploy or Argo CD.

Pipeline orchestrates the entire CI/CD flow — think Jenkins Pipelines or GitHub Actions workflows.

TestPlan is manual and automated test management — think TestRail or Zephyr.

PerfTest handles performance and load testing — comparable to JMeter or LoadRunner.

And **Governance** is software composition analysis and license compliance — Snyk or Black Duck territory.

All of these sit on top of Huawei Cloud's broader infrastructure — **SWR** for container images, **CCE** for managed Kubernetes, **ECS** for virtual machines — and we'll see each of those in action through the rest of the walk-through.

The reason this integration matters in practice: if you've ever spent a sprint debugging why your Jira-to-Bitbucket integration stopped passing commit messages through, or why a Jenkins-SonarQube plugin broke after an upgrade, you know the cost of stitching. CodeArts is essentially a bet that integration is worth more than best-of-breed. By the end of today, you should have a clear sense of whether that's the right bet for our team.

With that grounded — let me show you the agenda for the rest of the session."

▸ ADVANCE

Slide 3 — Agenda

> "There are four building blocks I want to land. First, the HE2E DevOps framework — that's the philosophy underneath everything. Second, the Phoenix Mall sample — that's where we trace a single feature from idea to production. Third, security configuration — operational, code, and continuous-delivery security. And fourth, permissions — how we structure roles for nine different teams.

> If you only remember one thing from each block by the end, that's a win. I'll mark the takeaways as we go."

▸ ADVANCE

Slide 4 — The HE2E DevOps Framework

> "HE2E stands for Huawei End-to-End. It's not a brand-new methodology — it's an operable, implementable agile framework that combines what's been learned from the industry with Huawei's own practices.

> Think of it as three connected stages.

> **Stage one — Planning and Design.** This is where service personnel or customers sit down with technical people and figure out what we're actually solving. Software solves problems and delivers value, not just delivers functions. Two key tools live here: impact mapping, which traces user requirements back to root causes; and user stories, which describe scenarios — not just isolated requirement items, but the whole picture in a tree structure.

> **Stage two — Iteration tracking.** This is the Scrum framework — sprint planning, daily standups, sprint reviews, retrospectives. The deliverables are the product backlog, sprint backlog, potential deliverable increments, and an issue list. Teams can also borrow from Kanban — visualise the value stream, identify bottlenecks, accelerate feedback loops.

> **Stage three — Continuous Delivery.** This is the engineering practice — the CI/CD pipeline. Code repositories, security control of code assets, concurrent development, version and baseline management. The pipeline connects static scanning, automated builds, automated tests at every stage, and automated deployment. It also handles artifacts and environments — dev, test, staging, production.

> The key idea on this slide is the callout: software doesn't exist to ship features, it exists to deliver value. HE2E keeps the team aligned from intent all the way to release.

> **For context — how this compares.** If you've used GitLab's DevOps platform or the Atlassian stack — Jira plus Bitbucket plus Bamboo — the philosophy is similar: one integrated suite where planning, code, build, test, and deploy all live in the same place rather than stitching together five separate vendors. The trade-off is the same too: you give up some best-of-breed depth in any single tool in exchange for friction-free integration across the whole loop."

▸ ADVANCE

Slide 5 — Meet Phoenix Mall

> "To make all of this concrete, the guide walks through a single sample project called Phoenix Mall.

> The story is: Company A is an automobile parts dealer. They want to boost sales by building an online store for parts. The mall manages members, parts, and orders, lets customers query products, and gives the company management visibility into what's happening.

> Two things to note.

> One — they use **Scrum**, two-week sprints. The Phoenix Mall has already been delivered through three sprints, and Sprint 4 is about to start. Sprint 4 has one new requirement: store query — letting users see the network of all stores.

> Two — that's the feature we'll trace end to end. Same feature, every stage of the loop. So when you see "store query" come up later, that's the connective thread."

▸ ADVANCE

Slide 6 — Solution Architecture

> "Architecturally, Phoenix Mall is five microservices.

> **Vote** — that's the web client. Users hit it through a browser, click 'like' on a part, and Vote writes that to Redis. Tech stack is Python and Flask, served by Gunicorn.

> **Result** — that's the management web UI. It shows aggregated stats — what's been liked, what's selling. Tech stack is Node.js and Express.

> **Worker** — a background batch process. It reads new records out of Redis and persists them into PostgreSQL. Tech stack here is interesting — the sample provides both .NET Core and Java implementations of the same thing, so you can switch by config.

> **Order Cache** — Redis, persisting client UI data.

> **Order Database** — PostgreSQL, the source of truth for the management UI.

> The flow, end to end, is: user clicks 'like' on Vote → Vote writes to Redis → Worker pulls from Redis → Worker writes to Postgres → Result reads Postgres and displays.

> Each of those five lives in its own container, gets built independently, gets pushed independently, and gets deployed together. That's the architecture in one breath."

▸ ADVANCE

Slide 7 — The Team

> "Four roles drive this. Keep these names in mind because they'll keep coming back.

> **Sarah** is the project administrator. She plans the project, sets up the team, configures permissions. Sarah does all the prep work.

> **Maggie** is the project manager. She owns delivery plans, breaks down stories, reviews and merges code.

> **Chris** is the developer. Chris is the doer of the loop — codes, builds, deploys, verifies. We'll see his name a lot.

> **Billy** is the tester. Billy writes test cases, runs them, raises bugs, signs off on quality.

> So as the slides progress and I say 'Maggie does X' or 'Chris configures Y' — these are the four characters in the play."

▸ ADVANCE

Slide 8 — Resource Planning

> "Before any code is written, three resources need to be in place — and they all need to live in the same region.

> First, **CodeArts itself**. Basic edition, five users, one-month subscription is enough for this practice.

> Second, **CCE — Cloud Container Engine**. Standard cluster, x86 architecture, tunnel network, nodes with at least 2 vCPU and 8 GB of RAM running EulerOS. Pay-per-use billing.

> Third — and this is an alternative, not in addition to — **ECS, an Elastic Cloud Server**. Ubuntu 16.04, 2 vCPU and 8 GB, with a 5 megabit elastic IP, and importantly, the security group needs to open ports 5000 and 5001 to the world.

> The reason both are listed: the practice supports **either** CCE or ECS deployment. Pick the one that matches your platform — both are billed once provisioned, so don't accidentally spin up both."

▸ ADVANCE

Slide 9 — The End-to-End Loop

> "Here's the full nine-step loop. This is the spine of the rest of the presentation — every slide from here forward zooms into one of these boxes.

> Step one, **Prepare** — Sarah creates the project and adds members.

> Step two, **Plan** — Sarah and Maggie work the requirements and the sprint.

> Step three, **Code** — Chris creates a branch, commits, opens a merge request.

> Step four, **Check** — Chris runs static scans before the merge.

> Step five, **Build** — Chris builds container images and pushes them to SWR, Huawei's image registry.

> Step six, **Deploy** — Chris deploys, either to CCE or to ECS.

> Step seven, **Test** — Billy creates the test plan, designs cases, and executes them.

> Step eight, **Pipeline** — Chris stitches check, build, and deploy into one automated pipeline.

> Step nine, **Release** — Sarah cleans up the pay-per-use resources at the end.

> Now we'll look at each of these in detail."

▸ ADVANCE

Slide 10 — Step 1–2: Prepare and Plan

> "Sarah's first job is straightforward. Log into CodeArts, click Create Project, pick the DevOps Full-Process template, name it Phoenix Mall.

> Then she creates IAM users for Maggie, Chris, and Billy, adds them to the project, and assigns roles — project manager, developer, tester. There's one specific permission tweak: developers need 'Create Environment' under CodeArts Deploy turned on.

> Now Maggie takes over for the planning. Requirements are managed using a mind map in the **Epic → Feature → Story → Task** hierarchy.

> She right-clicks the existing Phoenix epic and adds a new feature: **Store Network**. Under that feature, she adds a story: 'User can query network of all stores.' Priority high, severity critical.

> The story gets edited with a clear description in the user-story format — 'As a user, I want to query all stores so that I can select a proper store for services.' She uploads the actual store list as an attachment to the story so everyone has the same source of truth.

> Then she creates Sprint 4 — two weeks long — adds the story to it, assigns it to Chris, and breaks it into two child tasks: 'Frontend display - Add store network menu' and 'Background management - Add store network management module.'

> The callout matters: tracking through this isn't just for show. CodeArts gives you progress dashboards out of the box, and you can build custom reports for things like burndown, blockers, and so on.

> **How CodeArts Req compares.** Functionally this is the same territory as **Jira**, **Azure DevOps Boards**, or **Linear**. The Epic-Feature-Story-Task hierarchy is essentially Jira's Epic-Story-Subtask model with one extra layer. The mind-map view and the Scrum sprint board map directly to what you'd see in Jira's backlog and sprint views. The big upside over Jira specifically:

> requirements, branches, commits, scans, builds, deploys, and test cases are all linked natively — no Jira–Bitbucket–Jenkins plugin glue to keep alive."

▸ ADVANCE

Slide 11 — Step 3: Develop Code

> "Now Chris codes. The golden rule on this slide is the one we should all internalise: **never code on master.**

> The sample repo creates a default master branch. To keep master stable and continuously available, every feature lives on its own branch and gets reviewed before merging.

> So Chris does four things, in order.

> One, he creates a branch called Feature-Store, based on master, and explicitly associates it with the work item — that linkage is why CodeArts can later auto-update task status.

> Two, he edits two files in the repo: store-network.html — where he adds the list of branches — and index.html, where he adds the store network menu link. His commit messages reference the task ID — that pattern, 'fix #xxxxxx Frontend display - Add store network menu', is what closes the loop back to Req.

> Three, he opens a merge request from Feature-Store into master, and sets Maggie as the approver and merger.

> Four, Maggie reviews, leaves comments, approves the gate, and merges. As soon as the commit lands, the task in CodeArts Req auto-updates from In Progress to Resolved, with the commit visible in the work item's Associations tab.

> So you get reviewed code, traceability from work item to commit, and a stable master — all from one workflow.

> **How CodeArts Repo compares.** It's Git under the hood, so if you know **GitHub**, **GitLab**, or **Bitbucket**, you already know 90% of this. A 'Merge Request' here is the same thing **GitLab** calls a Merge Request and what **GitHub** calls a Pull Request. Approval gates, protected branches, code review comments — all conceptually identical. The differentiator is the deep tie-in: when Chris's

> commit message contains the task ID, Req auto-updates the work item state — that linkage normally takes a Jira/GitHub integration plugin."

▸ **ADVANCE**

Slide 12 — Step 4: Check Code

> "Static analysis happens through CodeArts Check. The motivation in the guide is honest: as Phoenix Mall got bigger, more issues piled up, fixes got expensive, and there were no unified coding standards. The fix was: standards plus continuous static scanning plus issue-fixing inside each sprint.

> The sample project ships **four preset check tasks**: one for Worker, one for Result, one for Vote, and one that scans JavaScript across the entire repo.

> Default rule sets are Java-only. So if you're scanning Python code — like Worker — you go into the task settings, open Rule Sets, click Refresh, and toggle Python on. That's it. Then click Execute Check.

> When the scan completes, the Issues tab lists every finding. Click 'Help' on any issue and you get a fix suggestion plus a direct link to the file and line in the repo. Chris's job is then to click through, fix the high-priority issues, and re-run the scan.

> The bigger point — and we'll come back to this when we talk about pipelines — is that this scan eventually becomes a **gate**. If the issue count breaches a threshold, the pipeline fails and the code never reaches prod.

> **How CodeArts Check compares.** Most teams in our space know **SonarQube** or **SonarCloud** — that's the closest analogue. Other peers are **Codacy**, **Code Climate**, **Snyk Code**, and **DeepSource**. The rule-set model, the per-language scanners, the inline fix suggestions, and the quality-gate concept are all standard SonarQube territory. If you've configured a Sonar quality gate before, you'll feel right at home here."

▸ ADVANCE

Slide 13 — Step 5: Build and Push to SWR

> "Now we package microservices into images. The reason matters: deployment failures often come from inconsistent environments. The classic example from the guide — JDK gets upgraded in dev, nobody updates the environment list, prod stays on the old version, things break. Containerising the app plus its environment together eliminates that whole class of problem.

> Phoenix Mall has **five preset build tasks**: one main build, one test-environment build, and three per-microservice builds for Worker, Result, and Vote.

> What does the main build task actually do? Quite a lot. It builds the Vote, Result, and Worker images. It generates Dockerfiles for Postgres and Redis on the fly using shell commands, then builds those images too. It pushes all five into SWR — Huawei's container registry — under an org you create called 'phoenix'. And it does the file-rewriting magic: replacing image versions in docker-compose-standalone.yml and in all the Kubernetes deployment YAMLs, then archiving those files to the release repo.

> Two automation options on the right.

> 'Run upon code commit' — every time someone pushes to master, the build kicks off automatically. That's continuous integration.

> 'Scheduled execution' — pick days and a specific time, e.g. every weekday at noon. Useful for nightly builds.

> When the build is done, you'll see five images in SWR — vote, result, worker, postgres, redis — and the deployment files archived in the release repo, all tagged with the build number. Ready to deploy.

> **How this compares.** CodeArts Build sits in the same space as **Jenkins**, **GitHub Actions**, **GitLab CI**, **CircleCI**, and **Azure Pipelines**. The 'preset task' concept is closest to a GitHub Actions reusable workflow or a Jenkins pipeline template. Where Build differs slightly is that it's more 'task-

> oriented' — you assemble named actions in a UI rather than writing a YAML file from scratch. You can still write declarative configs if you want to, but the visual model is the on-ramp.

> **SWR — Software Repository for Container — compares to Docker Hub, AWS ECR, Google GCR, Azure ACR, or Harbor.** It's a private registry with image-level access control. Same mental model as ECR: create an org or namespace, push images with a build number tag, pull them at deploy time."

▸ ADVANCE

Slide 14 — Step 6a: Deploy to CCE

> "There are two deployment paths. We'll cover CCE first, ECS next. Pick whichever matches your platform — most modern teams will go CCE.

> The sample preset is called **phoenix-cd-cce**. Four steps.

> One — clear the cluster. Before deploying, make sure no stale workloads are sitting in the phoenix-cce cluster from a previous run. Delete them with full resource release.

> Two — configure the deployment. Set the cluster name and the namespace, point the parameters at your build task and version 1.0.0.

> Three — Save and Deploy. If everything's wired up correctly, five workloads come up Running on the CCE Deployments page.

> Four — expose the services. The vote pod and the result pod each need a LoadBalancer with an EIP so users can hit them. You auto-create a load balancer, name it 'phoenix', auto-create the EIP, set vote's container port 80 and service port 5000, then update result with the same load balancer at service port 5001.

> Verify by browsing to http://EIP:5000 — the Phoenix Mall homepage shows up — and http://EIP:5001 — the dashboard shows up. Done.

> **How this compares.** CCE is Huawei's managed Kubernetes — it's the direct equivalent of **AWS EKS**, **Google GKE**, or **Azure AKS**. Same Kubernetes underneath, same kubectl-compatible API, same workload concepts. The CodeArts Deploy application that targets it is closer to **Argo CD** or **Spinnaker** in spirit — a deployment controller with environments, parameters, and rollback — but UI-driven rather than GitOps-style."

▸ ADVANCE

Slide 15 — Step 6b: Deploy to ECS

> "If you're not running Kubernetes, the alternative is deploying directly to a single host using Docker Compose. The sample preset for this is **phoenix-sample-standalone**.

> Three sub-steps.

> First — register the host with the project. Go to Settings, Basic Resources, Create Host Cluster — call it phoenix-hostgroup, OS Linux, direct connection mode. Then import your phoenix-ecs instance, authenticate with root and the password you set when you bought it, SSH on port 22. Connectivity check should turn green; if it doesn't, the guide has a troubleshooting reference.

> Second — pre-deploy dependencies. The sample needs Docker and Docker-Compose installed on the target. The phoenix-sample-predeploy application does that for you. You create a phoenix-env environment, import the host into it, then in the Install Docker action add two shell commands: 'docker -v' and 'docker-compose -v' to confirm versions are printing.

> Third — actual deploy. In phoenix-sample-standalone, set the source to the build task phoenix-sample-ci, environment to phoenix-env, fill in your SWR creds — server, username, password — as parameters, and Save and Deploy.

> Verify the same way: http://EIP:5000 for homepage, http://EIP:5001 for the dashboard.

> Same outcome as CCE — different path.

> **How this compares.** ECS is Huawei's VM service — direct equivalent of **AWS EC2**, **Azure Virtual Machines**, or **Google Compute Engine**. The deploy-by-shell-script pattern is what most of us already know from **AWS CodeDeploy**, **Octopus Deploy**, or **Ansible**-driven CD. If your team has been deploying to EC2 with CodeDeploy or running Ansible playbooks against VMs, this path will feel very familiar."

▸ ADVANCE

Slide 16 — Step 7: Manage Tests

> "Now Billy takes over. CodeArts TestPlan is where the test cycle lives — plans, designs, executes, and tracks quality.

> Four steps.

> Step one — create the plan. Billy creates 'Test Plan of Sprint 4', sets himself as processor, matches the plan period to the Sprint 4 dates, picks Manual Test mode, and confirms the linked requirement is the Sprint 4 store-query story. The plan starts in the New state.

> Step two — design cases. Billy clicks Design, picks the story, creates a test case called 'Store Query'. The case has three steps: open the Phoenix Mall homepage, click the Store Network menu, select City A. Each step has an expected result. As soon as cases are designed, the plan status moves from New to Designing.

> Step three — execute. Once Chris has finished coding the story, deployed it to test, and handed off to Billy, Billy creates a test suite called 'Execute Test Case of Sprint 4', adds the case, and runs it step-by-step in the actual test environment.

> Step four — the bug loop. If a step fails — say the page redirects to a 404 — Billy marks that step Failed, adds a clear note, fails the suite, and clicks the bug-create button. He titles the bug 'Store network page 404', assigns it to Chris. Chris fixes it. Billy re-runs the case. If it passes, the bug closes. If it fails again, the loop repeats.

> The TestPlan quality dashboard pulls all of this together — requirement coverage, defect counts, case pass rate, case completion rate. It's the single source of truth for whether we're ready to release.

> **How CodeArts TestPlan compares.** Closest analogues are **TestRail**, **Zephyr** (the Jira plugin), **qTest**, and **Xray**. Same concepts: test plans, test suites, test cases, test runs, defect linkage. The differentiator is again integration depth — when Billy creates a bug from a failed test step, it

> lands directly on Chris's board in Req with the failing step embedded. With TestRail plus Jira plus a plugin you can get there, but it's three things to maintain."

▸ **ADVANCE**

Slide 17 — Step 8: Pipelines

> "This is where everything comes together. A pipeline is what turns the previous steps from a sequence of manual actions into one automated, repeatable flow.

> The sample ships five preset pipelines. The main one — **phoenix-workflow** — is what we'll use.

> Out of the box it has three stages: Pipeline Source, Build, Deploy. Chris adds a fourth stage in between Source and Build: a Check stage. He clicks the plus icon, names the stage 'Check', adds a Job, and inside the job picks the phoenix-codecheck-worker task we set up earlier. So the pipeline now flows: source → check → build → deploy.

> Configuring the deploy task: he picks the build task to draw artifacts from, and the deploy parameters need to match what's configured on the deploy applications. Note: the sample includes both the CCE and ECS deploy tasks — keep the one you use, delete the other.

> Two configurations on the bottom that matter for production use.

> **Pass conditions on the Check stage.** Add a SystemPolicy condition. Now if the static scan finds more than the allowed number of issues, the pipeline fails — bad code can't reach prod.

> **Auto-trigger on commit.** Under Execution Plan, enable Trigger Settings, pick code commit, pick the master branch. Now every merge to master triggers the full pipeline. That's full continuous delivery.

> **How CodeArts Pipeline compares.** Direct equivalents are **Jenkins Pipelines**, **GitHub Actions workflows**, **GitLab CI/CD pipelines**, **CircleCI workflows**, and **Azure DevOps Pipelines**. The stage-and-job model is industry-standard. The 'pass conditions' on a stage are what GitHub Actions calls 'job-level conditions' or what Jenkins calls a 'quality gate'. The trigger-on-commit option is a webhook under the hood, same as `on: push` in GitHub Actions or a Jenkins SCM poll/webhook trigger. If anyone in your team is fluent in any one of these, the conceptual jump is small."

▸ ADVANCE

Slide 18 — Step 9: Release Resources

> "Quick but important. CCE and ECS are both billed pay-per-use. As long as they're running, the meter is running.

> Once the practice is over — or for us, once a project sunsets — Sarah's last job is to release the resources. Delete the ECS instance and its EIP. Delete the CCE cluster.

> The warning on this slide is real: **released resources cannot be recovered**. So before you delete, make sure you've kept anything worth keeping — artifact archives in the release repo, screenshots of dashboards, exported logs.

> This is the kind of step that gets forgotten and shows up in a finance review later. Don't let it."

▸ ADVANCE

Slide 19 — Configuring CodeArts Security

> "We've finished the operational walk-through. The next big block is security. The guide breaks it into three lanes — and we should adopt all three by default, not pick and choose.

> **Operational security.** Three-layer permission control — tenant, project, instance — all assigned by least privilege. CTS, Cloud Trace Service, is enabled to collect and store every operation record for compliance, audit, and troubleshooting. IP whitelists on Repo and Artifact reduce brute-force and DDoS exposure. And Repo lets you add a visitor watermark to source pages, which deters source code leaks.

> **Code security.** Access tokens, with a maximum 1-year validity, replace passwords whenever you're granting third-party access — and they're shown only once at creation, so they have to be stored properly. Deploy keys are read-only — that's what build agents should use to clone, never a user's full credentials. Protected branches mean master can only accept code through merge requests, and only authorised roles can push. Repo visibility lets you set private, public-to-project, public-to-tenant, or fully public per repo. Commit rules block specific commit patterns.

> **Continuous-delivery security.** Private parameters in Build, Deploy, Pipeline, and TestPlan are encrypted at rest, decrypted only at runtime, and invisible in logs — that's where API keys, passwords, certificates go. Host security groups: when CodeArts Deploy reaches your hosts through the default agent pool, you whitelist the agent's public IP in your security group so only that path works.

> The default posture is in the callout: protect master, store secrets as private params, enable CTS, deploy keys for read-only repo access from agents.

> **How these compare.** A couple of mappings worth flagging.

> **CTS — Cloud Trace Service** is Huawei's audit-log service. Direct equivalent of **AWS CloudTrail** or **Azure Activity Log** — every API call and config change captured in a queryable record.

> **Private parameters** are conceptually identical to **GitHub Actions Secrets**, **GitLab CI/CD Variables (masked + protected)**, **Jenkins Credentials**, or **HashiCorp Vault** dynamic secrets — encrypted at rest, decrypted only at runtime, redacted in logs.

> **Protected branches and merge approval rules** are the same model as **GitHub branch protection rules** or **GitLab push rules** — same concept, same enforcement points.

> **Deploy keys** are literally a Git/GitHub concept. Same pattern, same security property: read-only credentials scoped to one repo and used by automation, not humans."

▸ ADVANCE

Slide 20 — Concrete Security Checklist

> "Here's the do-this-on-day-one version. Eight items, in priority order.

> Three-layer permissions assigned by least privilege.

> CTS audit logs enabled across every CodeArts service.

> IP whitelists on Repo and Artifact.

> Watermark Repo views — there's a real cost to source leaking and this is a one-toggle deterrent.

> Access tokens with TTLs — never let them be open-ended, never share user credentials.

> Deploy keys, read-only, for CI access.

> Master branch protected — pushes are MR-only, merges by authorised roles only.

> All secrets — without exception — stored as private parameters, never in plain config.

> If we get all eight in place before we onboard a single feature team, we'll have set ourselves up correctly. Adding security after the fact is always more painful than building it in."

▸ **ADVANCE**

Slide 21 — Permissions for Nine Teams

> "The guide ships a worked example of permission configuration for an IPD project — Integrated Product Development — with **nine different teams**. The principle is: build on system roles, copy-and-adjust where you need a variant, save the result as a template you can reuse on the next project.

> Walking left to right, top to bottom.

> **R&D team.** R&D leader gets branch management plus code review. Developer gets view, branch creation, and merge requests.

> **Middle-end team.** This is the platform team that builds reusable services for the rest of the org. Their roles are direct copies of R&D leader and Developer.

> **Architecture team.** Architecture leader equals R&D leader; system engineers equal developers. The leader extends to managing repositories more broadly.

> **Test team.** Test manager gets full TestPlan plus view-only on Repo — they need view access for white-box testing but they shouldn't be able to edit code. Tester gets case management and mind maps.

> **PO team — Product Owner.** Product leader maintains raw requirements, initial requirements, user stories, and version iterations. Product manager retains defaults.

> **PM team — Project Management.** Owns test plans and project tracking, but no code edit access.

> **Big data team.** Big data leader, developer, and PM are clones of the equivalent R&D and Product roles, scoped to data work.

O&M team. O&M leader has full pipeline rights. O&M engineer can build, deploy, execute. On-call engineer gets read-only — they shouldn't be making changes during incidents, just observing.

DBA team. DBA leader gets full Pipeline access plus the privilege-config permission for SQL automation. Members copy that role.

Nine teams, one consistent pattern.

How this compares. The role-based access model is conceptually similar to **GitHub Organisations + Teams + repository roles** or **GitLab Groups + custom role permissions**. The 'system role + custom role + copy role + save as template' pattern is closest to **Azure DevOps' security groups with inheritance**. If you've configured org-level permissions in any of those tools, you'll recognise the model. The CodeArts difference: the permission scope cuts across all eight services in one screen rather than configuring it separately in each tool."

▸ ADVANCE

Slide 22 — Permission Principles

> "Three principles to guide our rollout.

> One — **start from system roles.** CodeArts ships project manager, developer, tester. Don't reinvent. Extend.

> Two — **use 'Copy Role'.** When you need a variant — Big Data Developer is just a Developer scoped to the data domain — copy and adjust the deltas. You'll have ten roles in twenty minutes instead of ten days.

> Three — **save as templates.** Once a role set is good, save it as a permission template and apply it to the next project. This is genuinely the biggest time-saver across an organisation.

> One edge case worth flagging — and it catches teams out. Test managers and testers don't get Repo edit access. But they do need view access to do white-box testing. The defaults handle this correctly. Don't strip Repo entirely from the test team or you'll break their workflow."

▸ ADVANCE

Slide 23 — Key Takeaways

> "If we agree on these eight things in this room, our CodeArts implementation succeeds.

> One — adopt the HE2E loop end to end. Don't pick and choose stages. The value comes from the connection.

> Two — branch and MR is mandatory. Master is sacred. Every change goes through a reviewed merge request.

> Three — static checks live inside the pipeline, with pass conditions. Bad code shouldn't be able to reach prod.

> Four — containerise everything. Same image, every environment. Eliminates 'works on my machine'.

> Five — auto-trigger on commit. Builds and pipelines run on push to master. No manual runs as part of the normal flow.

> Six — security on by default, day one. CTS, whitelists, protected branches, private parameters.

> Seven — role templates for every team. Save once, reuse forever.

> Eight — release pay-per-use cleanly. We always tear down what we no longer need.

> Each of these is straight out of the practice guide. They're not nice-to-haves — they're the difference between CodeArts working for us and us working for CodeArts."

▸ ADVANCE

Slide 24 — Closing / Q&A

> "That's the end of the walk-through. So — what's next? I'd like us to leave this meeting with three concrete actions.

> First, agree on whether we're going CCE-first or ECS-first for our deployment target.

> Second, identify which team in our org maps to the R&D leader, developer, tester, and PM roles — names against roles, this week.

> Third, pick a single feature in our existing backlog to be our 'Sprint 4 store query' — the one we'll trace end to end as a pilot.

> Once those three are settled, I'll come back to this group with a concrete implementation plan and timeline.

> Happy to take questions now."